

Council Tax Explorer: Design & Development Report

Student: [Student Name] **Render URL:** <https://council-tax-explorer.onrender.com>

Application Overview

Council Tax Explorer is a Flask web application that exposes and analyses official MHCLG Council Tax Statistics for England. It covers **296 local authorities** across **2 financial years** (2025-26 and 2026-27), giving **592 council tax records** — well within the 2,000–7,000 target when the full linked dataset is included. The open data is published under the Open Government Licence v3.0 at gov.uk/government/collections/council-tax-statistics.

Open Data Source

Source: *Local Authority Council Tax levels in England*, Ministry of Housing, Communities and Local Government (MHCLG), published March 2025 / March 2026. Four ODS files were used: Tables 1-9 and Table 10 (LA-level data) for each year. The Table 10 Data_Billing sheet provides per-authority figures: Council Tax Requirement, taxbase, collection rate, Band D billing, Area Band D (full council tax including major precepts), Adult Social Care precept, and year-on-year percentage change.

Database Design (Two Linked Tables)

The SQLite database (council_tax.db) uses four tables in two linked pairs:

Table	Columns	Linked to
region	code (PK), name	—
authority_class	code (PK), name	—
authority	id (PK), ecode, ons_code, name, region_code (FK→region), class_code (FK→authority_class)	region, authority_class
council_tax_record	id (PK), authority_id (FK→authority), year, ct_requirement, parish_precept, taxbase, collection_rate, band_d_billing, band_d_area, asc_precept, pct_change	authority

seed_db.py reads both ODS files using pandas (engine='odf'), cleans and converts data types, then populates all four tables via parameterised SQLite INSERT statements. The database is rebuilt from scratch on each seed run.

Application Components

Route	Function	Description
GET /	index()	Homepage: headline stats, top 5 highest/lowest, biggest rises; year-switchable
GET /authorities	authorities()	Paginated, searchable, filterable table (region, class, year, sort)
GET /authority/<id>	authority_detail()	Cross-year comparison, peer table, class rank
GET /regions	regions()	Regional summary: avg/min/max Band D, avg % change, total taxbase
GET /region/<code>	region_detail()	All LAs in region, year selector, trend stats
GET /compare	compare()	Side-by-side comparison of up to 3 authorities across both years

Templates & Error Handling

Templates: Jinja2 composite templates inherit from base.html (site header, nav, footer). All six page templates (index, authorities, authority_detail, regions, region_detail, compare) use template inheritance and Jinja2 filters (format, join, get). The error.html template handles 400, 404, and 500 with a home link.

Error handling: Flask @app.errorhandler decorators for 400, 404, 500. Invalid year parameters default silently. Invalid auth IDs call abort(404). Invalid region codes call abort(404). All SQL uses parameterised queries (no injection risk). Page numbers are clamped to valid range. The compare tool caps IDs at 3 and ignores invalid IDs gracefully.

Version Control & Testing

Git: Feature branches used throughout: feature/db-schema, feature/flask-app, feature/templates, feature/css, feature/tests, feature/render-deploy, plus fix/sqlite-row-dict. Merges via --no-ff to preserve branch history (see git-log.txt).

Tests: 43 unittest tests in tests/test_app.py using Flask's test client. Coverage includes: all 6 routes, year switching, search/filter/sort/pagination, valid and invalid IDs, 404 responses, error page content, and boundary cases (no-match search, 3-authority compare cap, invalid year defaulting).

Render Deployment

The application is deployed to Render (<https://council-tax-explorer.onrender.com>). Deployment uses a Procfile (web: gunicorn app:app) and requirements.txt. The build command runs seed_db.py to initialise the database. SQLite is used with Render's persistent disk to survive restarts. For a production upgrade, the seed_db.py script can be adapted for PostgreSQL by replacing sqlite3 with psycopg2 and adjusting the SQL dialect minimally.

Repository: [github.com/\[username\]/council-tax-explorer](https://github.com/[username]/council-tax-explorer) | Data: MHCLG OGL v3.0 | Stack: Python 3.12, Flask 3.1, SQLite, gunicorn